# Capstone Project Documentation

Name : L ISHWAR

Roll Number :  2329125

Batch : 2023-2027

Course: Agentic AI

## Title : Code Review Agent

This document presents the design, implementation, deployment, and evaluation of a Code Review Agent developed as a capstone project. The solution uses retrieval-augmented generation, memory-aware conversation flow, tool-based augmentation, and a production-friendly web interface for practical usage by developers.

# 1. Problem Statement

Code review quality often varies because reviewers are constrained by time, context switching, and uneven expertise across security, performance, testing, and maintainability concerns. As a result, teams may miss critical defects such as edge-case failures, weak validation, poor error handling, and scalability issues until late stages of development or production incidents. Developers, especially students and early-career engineers, also struggle to produce consistent, actionable review comments and to justify recommendations with clear reasoning.

The goal of this project is to build an AI-based assistant that can provide structured, grounded code-review guidance. The assistant should respond with high faithfulness, avoid unsupported claims, and adapt to different query types, including memory follow-ups and requests for latest references. It should be usable through a simple interface and deployable in a real cloud environment.

## Objectives

1. Build a domain-specific assistant for software code review guidance.

2. Use at least 10 curated knowledge documents to support retrieval-based answers.

3. Add conversation memory to support contextual follow-up questions.

4. Include a self-reflection/evaluation loop to reduce hallucinations.

5. Add one external tool (web search) for current references such as documentation and advisories.

6. Deploy the solution with a user-facing interface.

# 2. Solution and Features

## 2.1 Architecture Overview

The assistant is implemented using a LangGraph StateGraph pipeline with nodes for memory, routing, retrieval, tool execution, answer generation, evaluation, and save-to-history. The route decision node classifies queries into retrieval, memory-only, or tool-assisted paths. Retrieved or tool-generated context is passed to the answer node, which is constrained to grounded responses. A faithfulness evaluator can trigger retries when quality thresholds are not met.

## 2.2 Core Features

A. Retrieval-Augmented Generation (RAG): The system embeds domain documents and retrieves top relevant context chunks for each question. This keeps answers specific and aligned with known knowledge.

B. Conversation Memory: The system stores recent dialogue turns with thread-based memory so follow-up queries can reference earlier responses in the same session.

C. Tool Use (Web Search): The assistant can perform web lookup for latest documentation, release notes, and security advisories when the user asks current-information questions.

D. Self-Evaluation Loop: The assistant estimates answer faithfulness and retries generation if quality is below threshold, improving reliability.

E. Deployment UI: A Streamlit chat app provides an accessible frontend with session controls, source display, and contextual responses.

## 2.3 User Flow

1. User enters a code-review question in the Streamlit chat interface.

2. Router decides the path: retrieve, memory-only, or tool.

3. Relevant context is gathered from KB and/or tool output.

4. Answer node generates a grounded, structured response.

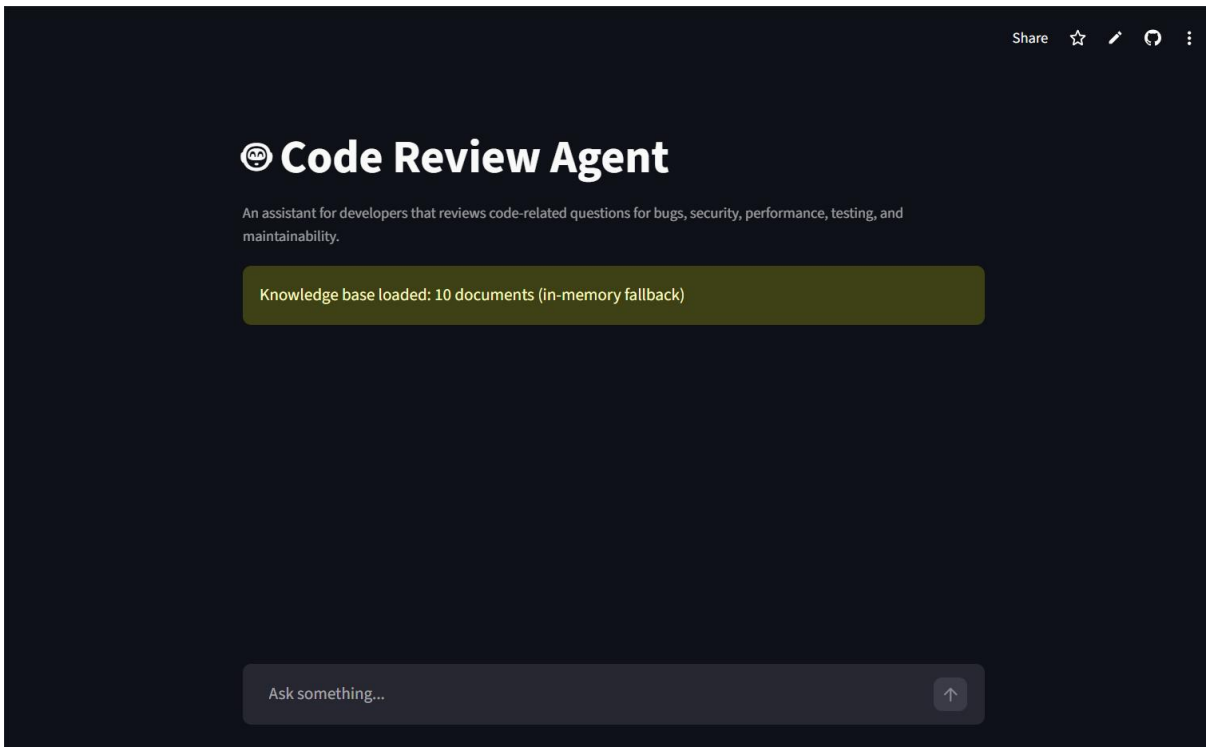
5. Eval node checks quality and retries if needed.

6. Final answer is shown and stored in conversation memory.

## 3. Screenshots

### Screenshot List (Suggested)

1. Application Home Screen (Streamlit UI loaded successfully).



## About

An assistant for developers that reviews code-related questions for bugs, security, performance, testing, and maintainability.

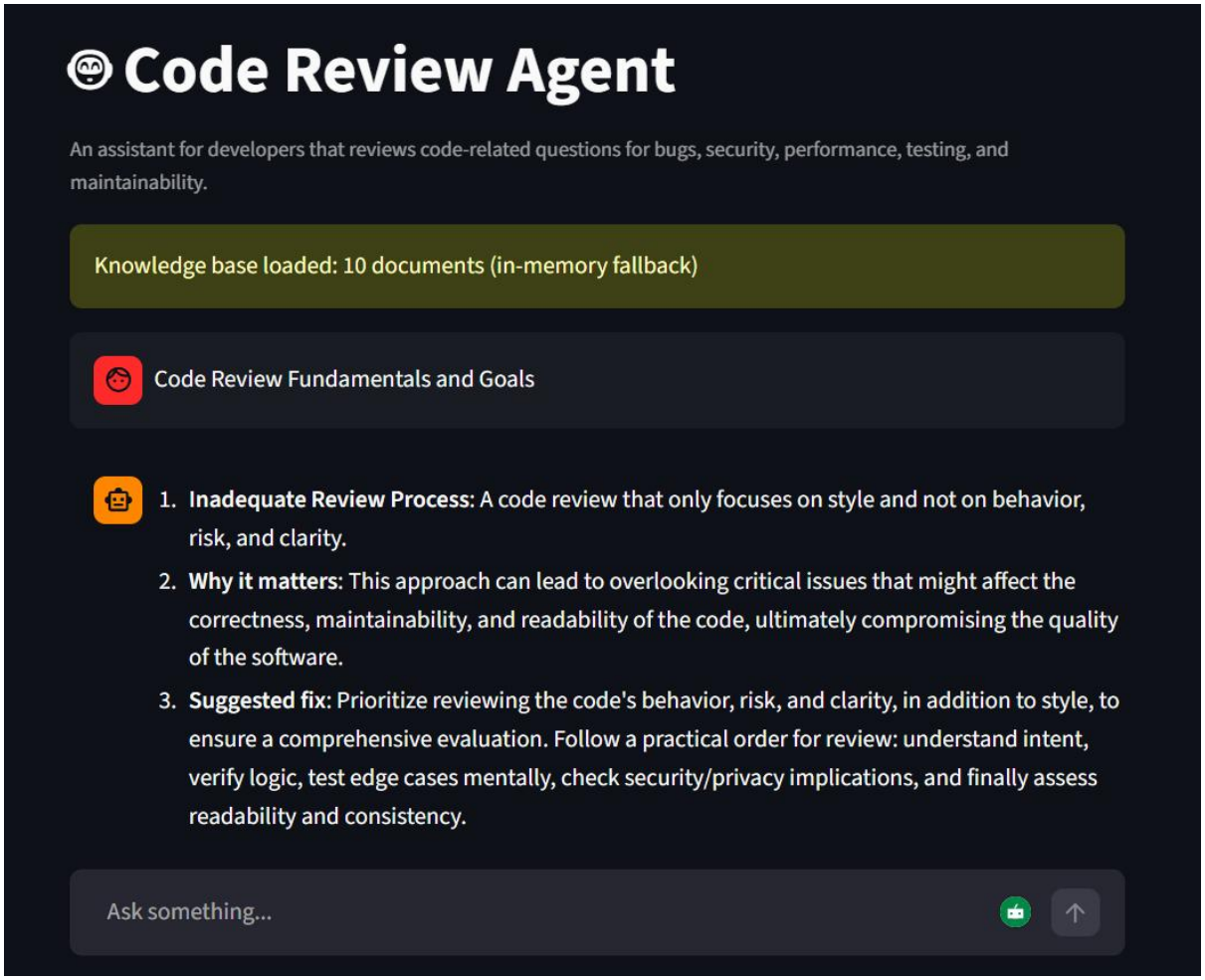
Session: 7bfd2003

---

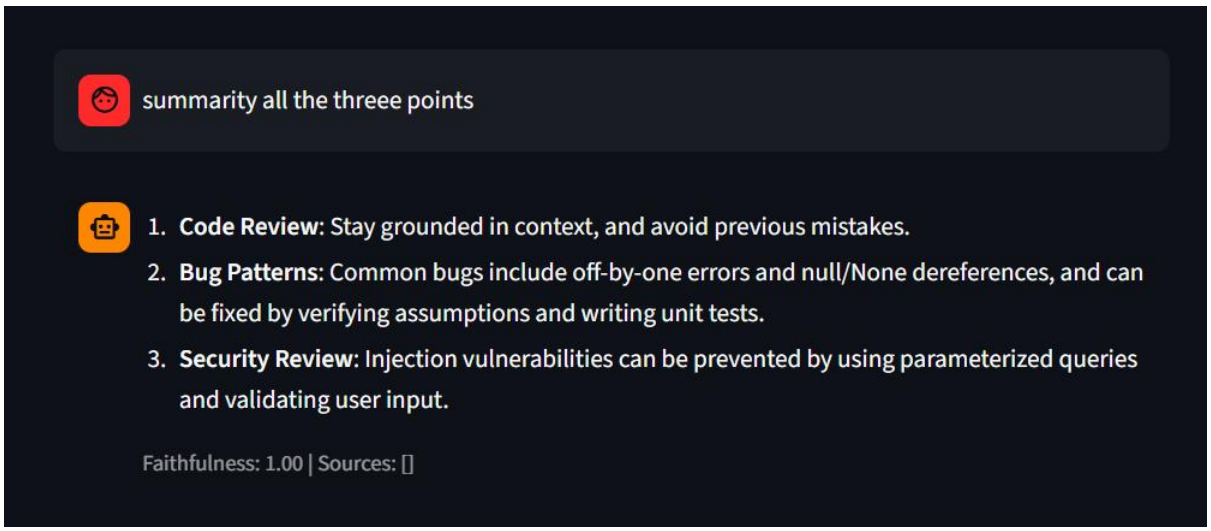
Topics covered:

- Code Review Fundamentals and Goals
- Bug Patterns in Everyday Code
- Security Review Basics for Developers
- Performance and Scalability Review Heuristics
- Readability, Naming, and Maintainability
- Error Handling and Observability
- Testing Strategy for Review Confidence
- API and Contract Review Checklist
- Database and Data Integrity Review

2. Normal Retrieval Example (question and grounded answer).




3. Memory Follow-Up Example (question referencing previous response).




## 4. Tech Stack

Programming Language: Python

LLM Provider: Groq (Llama 3.3 70B Versatile)

Orchestration: LangGraph

Core Prompt/Message Layer: LangChain Core

Embeddings: SentenceTransformers (all-MiniLM-L6-v2)

Vector Store: ChromaDB (with in-memory fallback for cloud compatibility)

External Tool: DDGS web search

Frontend/Deployment: Streamlit

Environment Management: python-dotenv

### Why This Stack

This stack provides a practical balance between rapid prototyping and production readiness. LangGraph enables explicit control over agent behavior. SentenceTransformers + retrieval provide grounded context. Streamlit simplifies UI and deployment. The fallback retrieval mode improves robustness on cloud runtimes with package constraints.

## 5. Unique Points

1. Deterministic + LLM Routing: Rule-based trigger checks are combined with LLM routing for improved reliability on memory and tool-intent queries.

2. Grounded Answer Contract: The response generator enforces a strict context-only rule and structured output format (issue, impact, fix).

3. Cloud-Robust Retrieval: If Chroma fails in cloud runtime, the app automatically switches to in-memory semantic retrieval, ensuring service continuity.

4. Quality Gate: Faithfulness scoring introduces a feedback loop to reduce hallucinated or weakly grounded responses.

## 6. Future Improvements

1. Hybrid Retrieval: Combine BM25 with vector search for better precision on keyword-heavy technical queries.

2. Better Evaluation Pipeline: Add automated batch eval with richer metrics and benchmark suites beyond faithfulness.

3. File/Code Input Mode: Allow users to paste code snippets or upload files, enabling line-level review suggestions.

4. Role-Specific Modes: Add tailored modes for backend, frontend, data engineering, and security reviews.

5. Citation UX: Show exact source snippet references per recommendation for stronger auditability.

6. CI Integration: Integrate agent checks into pull request workflows for team-scale adoption.

## 7. Conclusion

The Code Review Agent demonstrates a practical agentic AI system that combines retrieval, memory, tool use, and quality gating in a deployable Streamlit application. It addresses real review pain points by providing grounded, structured, and actionable feedback. With future enhancements in retrieval quality, code-aware input handling, and CI integration, the solution can evolve into a strong productivity assistant for software teams.